

Matriz de Decisión General

RF1 Recuperar y utilizar el inventario, las prioridades y los supuestos elaborados en el Hito 1.

Ver cuadro comparativo [acá](#).

RF2 Definir un conjunto de criterios técnicos y operacionales para comparar modelos NoSQL.

- 1. Capacidad y volumen de datos: puede manejar el volumen de datos acorde a lo que el módulo necesita?**
 - a. ¿Puede manejar el volumen de datos de los logs constantes?
 - b. puede manejar gran cantidad de usuarios esporádicos?
- 2. Performance: ¿Qué tan veloz es la tarea necesaria?**
 - a. ¿Puede ver el estado actual de un partido de forma (casi) instantánea?
 - b. ¿Qué tan rápido puede informar los ganadores?
 - c. ¿Qué tan rápido puedes actualizar blogs minuto a minuto?
- 3. Consistencia: En qué medida puede asegurarse que la información sea consistente?**
 - a. El estado del partido es verídico?
 - b. La cuenta y puntaje del usuario es acorde para todas las personas de un mismo grupo?
 - c. ¿Es consistente el ganador?
- 4. Escalabilidad y almacenamiento: Como se almacena, es escalable?**
 - a. Su método de almacenamiento, es costo efectivo?
 - b. es óptimo para manejar los datos globalmente?
 - c. ¿Cómo se utiliza?
- 5. Capacidad de consulta y modelado: ¿Cómo maneja consultas?**
 - a. ¿Puede formar relaciones que faciliten el desarrollo?

b. ¿Cómo es su optimización durante las consultas?

RF3 Asignar una ponderación explícita a cada criterio definido; la suma total deberá ser 100%.

1. Capacidad y volumen de datos ----- 20%
2. Performance ----- 25%
3. Consistencia ----- 20%
4. Escalabilidad y almacenamiento ---- 20%
5. Capacidad de consulta y modelado - 15%

RF4 Evaluar los seis modelos NoSQL trabajados en clase para cada necesidad de datos o subsistema priorizado.

- **Cassandra:** utilizado para manejar la gran cantidad de logs generados durante los partidos.
- **Redis:** haciendo uso de su velocidad y manejo de datos en tiempo real para estadísticas y valores, se utilizará para manejar los partidos y sus valores mientras suceden. Una vez terminado el partido, anota los datos en MongoDB y los libera del caché.
- **MongoDB:** gestiona los partidos en su forma estática, mientras que Redis maneja los valores cuando los partidos suceden en tiempo real.
- **IRIS:** va a manejar el almacenamiento ACID de los grupos. Nunca vas a querer estar en otro grupo que no sea el tuyo y vas a querer tener acceso consistente a las comparativas con tus compañeros. Asimismo, sus cualidades de sharding permiten distribuir los grupos según región o país.
- **Neo4j:** se utiliza para guardar las predicciones que generan los usuarios, estableciendo una relación directa entre un usuario y su predicción. Dichas predicciones nunca escalan más allá de la cantidad de partidos totales × usuarios.
- **InfluxDB:** se utilizará para almacenar métricas y estadísticas históricas que cambian a lo largo del tiempo, por ejemplo cantidad de usuarios conectados, eventos por minuto o evolución de estadísticas durante los partidos.

RF5 Construir una matriz de decisión que permita visualizar criterios, ponderaciones,

alternativas, valoración y resultado.

Para construir la matriz mencionada, utilizaremos la plantilla básica mencionada en la teoría, pero la aplicaremos a las funcionalidades técnicas formuladas, de esa forma podemos evaluar, ponderar y generar un resultado para cada criterio

1) Logs generados durante el partido

DB	Volumen	Velocidad	Consistencia	Escalabilidad	Consultas
MongoDB	★★★★	★★★★	★★★★	★★★★	★★★★
Neo4j	★★	★★	★★★★	★★	★★
Redis	★★★	★★★★★	★★★★	★★★★	★★★
Cassandra	★★★★★	★★★★★	★★★	★★★★★	★★★
IRIS	★★★	★★★★	★★★★	★★★	★★★★★
InfluxDB	★★★	★★★★	★★★	★★★★	★★★★

Cassandra. Es la que mejor responde a la gran cantidad de logs generados constantemente durante los partidos.

2) Partidos y valores en tiempo real

DB	Volumen	Velocidad	Consistencia	Escalabilidad	Consultas
MongoDB	★★★★	★★★★★	★★★★	★★★★	★★★★
Neo4j	★★	★★	★★★★	★★	★★
Redis	★★★	★★★★★	★★★★	★★★★	★★★★★

Cassandra	★★★★	★★★★	★★★	★★★★★	★★★
IRIS	★★★★	★★★★	★★★★	★★★★	★★★
InfluxDB	★★★	★★★★	★★★	★★★★	★★★★

Redis. Su velocidad y manejo de datos en tiempo real lo hacen adecuado para mantener los valores del partido mientras sucede. Cuando termina, los datos pasan a MongoDB y se liberan del caché.

3) Grupos y puntajes

DB	Volumen	Velocidad	Consistencia	Escalabilidad	Consultas
MongoDB	★★★★	★★★★★	★★★★	★★★★	★★★★
Neo4j	★★	★★	★★★★	★★	★★
Redis	★★★	★★★★★	★★★★	★★★★	★★★
Cassandra	★★★★	★★★★	★★★	★★★★★	★★★
IRIS	★★★★	★★★★	★★★★★	★★★★★	★★★★★
InfluxDB	★★★	★★★★	★★★	★★★★	★★★★

IRIS. Se prioriza el almacenamiento ACID y que los usuarios tengan acceso consistente a su grupo, sus puntajes y las comparativas con sus compañeros.

RF6 Seleccionar un modelo principal para cada necesidad priorizada y registrar, como mínimo, una alternativa considerada.

- **Logs en tiempo real** → Cassandra. Alternativa considerada: InfluxDB. Cassandra se selecciona por su capacidad para manejar grandes volúmenes de escrituras y crecimiento sostenido; InfluxDB sería una alternativa si se priorizara el componente temporal de los logs.
- **Partidos en tiempo real** → Redis. Alternativa considerada: Cassandra. Redis se selecciona por su velocidad, baja latencia y manejo de datos transitorios en tiempo real. Cassandra podría soportar el volumen y las escrituras, pero Redis resulta más adecuado para mantener el estado actual del partido.
- **Persistencia y partidos estáticos** → MongoDB. Alternativa considerada: IRIS. MongoDB gestiona los partidos una vez finalizados mediante documentos flexibles. IRIS podría almacenarlos con mayor consistencia transaccional, pero sería innecesario para información que ya no cambia constantemente.
- **Manejo de grupos** → IRIS. Alternativa considerada: Neo4j. IRIS se selecciona por su almacenamiento ACID y la necesidad de mantener consistentes los grupos, puntajes y comparativas entre usuarios. Neo4j representaría bien las relaciones usuario-grupo, pero la prioridad en este caso es la consistencia.
- **Guardado de predicciones** → Neo4j. Alternativa considerada: MongoDB. Neo4j permite establecer directamente la relación entre un usuario, su predicción y el partido. MongoDB podría almacenar las predicciones como documentos, pero representa estas relaciones de forma menos natural.
- **Métricas y estadísticas históricas** → InfluxDB. Alternativa considerada: Cassandra. InfluxDB se selecciona por estar orientada a series temporales y permitir analizar la evolución de estadísticas, usuarios conectados y eventos por período. Cassandra también soportaría el gran volumen histórico, pero está menos especializada en consultas y agregaciones temporales.

RF7 Justificar cada selección con base en características del modelo, requisitos del Fixture 2030 y trade-offs aceptados.

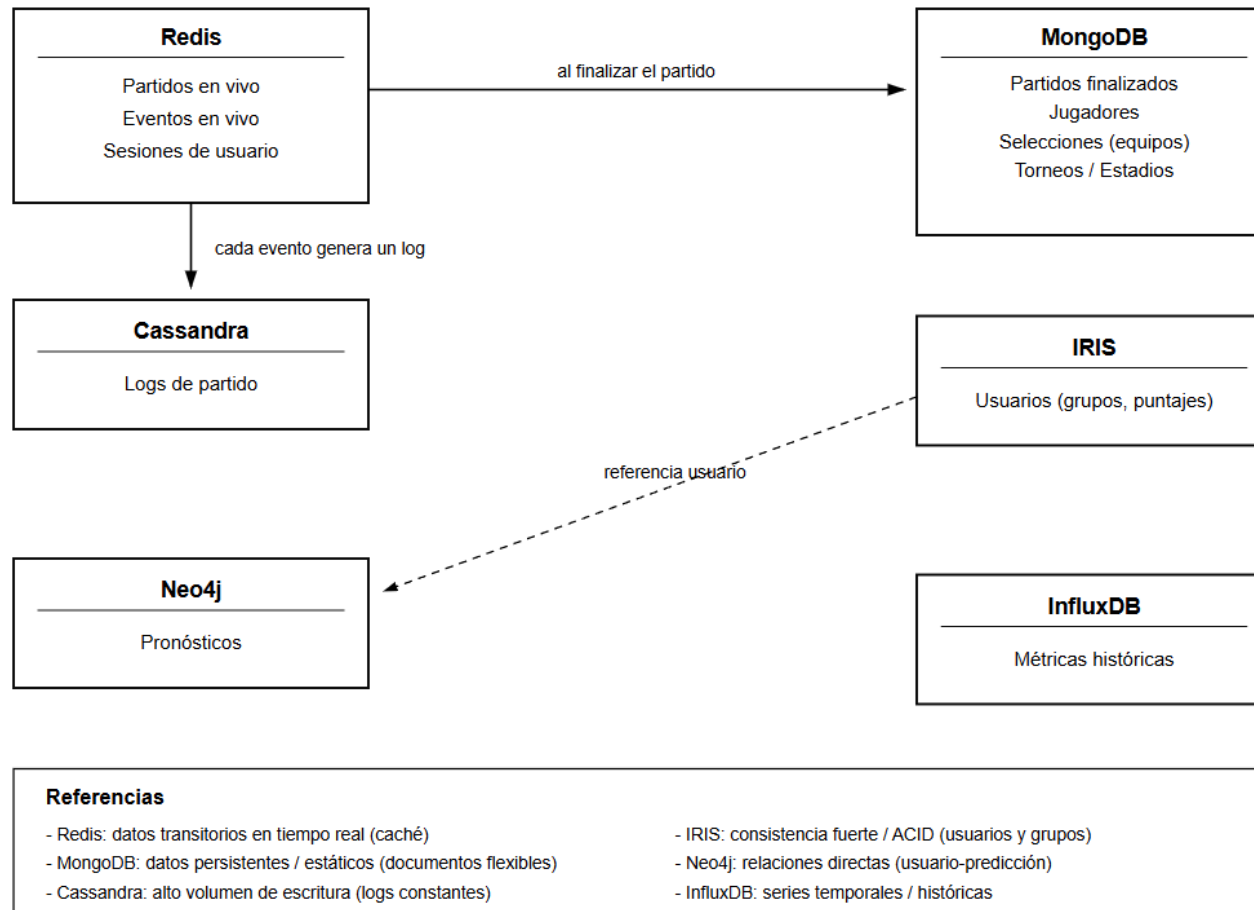
- **Logs en tiempo real:** Cassandra fue seleccionada porque está orientada a escrituras intensivas, distribución horizontal y alta disponibilidad. Estas características resultan adecuadas para almacenar el flujo constante de logs generado por los partidos y sus eventos. Como trade-off, se acepta una consistencia configurable y un modelado orientado a consultas, que limita las consultas improvisadas. InfluxDB sería más conveniente para análisis puramente temporales, pero Cassandra ofrece mayor flexibilidad para logs con diferentes estructuras.
- **Partidos en tiempo real:** Redis permite conservar en memoria el estado actual de partidos y eventos, ofreciendo la baja latencia necesaria para responder en menos de 100 ms durante picos superiores a 100.000 solicitudes por segundo. El trade-off aceptado es su mayor costo de memoria y el riesgo de pérdida de datos si ocurre una falla antes de persistirlos.

Este riesgo se reduce transfiriendo periódicamente la información definitiva a MongoDB. Cassandra ofrecería mayor durabilidad, pero no la misma velocidad para consultar el estado vigente.

- **Persistencia y partidos estáticos:** MongoDB permite representar cada partido finalizado como un documento que incluya equipos, estadio, resultado y eventos relacionados, evitando consultas con múltiples JOIN. Su esquema flexible también facilita incorporar nuevos atributos sin realizar migraciones complejas. Como trade-off, se acepta cierto grado de duplicación y desnormalización de datos. IRIS ofrecería mayor control transaccional, pero esa capacidad no resulta prioritaria para información histórica que, una vez validada, cambia con poca frecuencia.
- **Manejo de grupos:** IRIS fue seleccionado por sus capacidades transaccionales ACID, necesarias para que la composición de los grupos, sus puntajes y las posiciones observadas por los usuarios permanezcan consistentes. También permite distribuir la información para acompañar la operación del Fixture 2030 en diferentes regiones. El trade-off es una mayor complejidad operativa y la incorporación de un motor especializado. Neo4j representaría naturalmente las relaciones entre usuarios y grupos, pero el requisito principal es actualizar correctamente los puntajes y no recorrer relaciones complejas.
- **Guardado de predicciones:** Neo4j permite representar directamente las relaciones entre usuarios, partidos y pronósticos mediante nodos y vínculos, facilitando consultas que recorran esas conexiones. El volumen permanece limitado por la cantidad de usuarios multiplicada por los partidos disponibles. Como trade-off, se acepta mantener un motor adicional y una mayor complejidad de operación. MongoDB sería más sencillo si los pronósticos sólo se consultaran por usuario o partido, por lo que la conveniencia de Neo4j deberá validarse según las consultas implementadas en los próximos hitos.
- **Métricas y estadísticas históricas:** InfluxDB fue seleccionado porque está orientado a series temporales y permite almacenar datos asociados a marcas de tiempo, realizar agregaciones por período y analizar su evolución. Esto se ajusta a métricas como usuarios conectados, eventos por minuto y estadísticas generadas durante los partidos. Como trade-off, se acepta que no sea adecuado para documentos generales ni relaciones complejas. Cassandra soportaría un volumen histórico similar, pero requeriría mayor trabajo para implementar consultas y agregaciones temporales.

RF8 Elaborar un mapa de persistencia preliminar que relacione las necesidades del Fixture 2030 con las decisiones tomadas.

Mapa de persistencia preliminar - Fixture 2030



RF9 Identificar preguntas, riesgos o validaciones que deberán resolverse antes o durante los hitos de implementación.

- Definir cómo será la persistencia de los datos de Redis en MongoDB
- Definir cuál va a ser el disparador para la persistencia de Redis a MongoDB (final de partido, timer, manual, etc.)
- Pensar cómo será el manejo en el caso de algún error entre el caché de Redis y la persistencia de MongoDB
- Límite de escalabilidad de Neo4J en caso de que la cantidad de usuarios supere cierto punto
- Debatir los costos de usar de 6 motores distintos
- Backup/Alternativas en caso de ser necesarios
- Decidir el límite que se tendrá en almacenamiento de logs
- Definir cuestiones de seguridad (datos que necesiten ser encriptados, privacidad, etc.)